

Quizzy: Multi-User Exam-Prep Platform

Student Details

Name: Bhoomi Agrawal

Roll Number: 23f3003038

Email: 23f3003038@ds.study.iitm.ac.in

1. Introduction

Quizzy is a web-based exam-preparation platform designed to support both instructors (administrators) and learners (students) through interactive quizzes, performance tracking, and scheduled notifications. It offers a secure, role-based environment for content creation, assignment, and analysis of quiz activity.

2. Objectives

- **Streamlined Content Management:** Enable administrators to create and organize subjects, chapters, quizzes, and questions via a RESTful API.
- **Interactive Quiz-Taking:** Provide students with a responsive interface for attempting timed quizzes and viewing detailed feedback.
- **Asynchronous Features:** Automate reminder emails for upcoming deadlines and export of quiz histories using background tasks.
- **Performance Tracking:** Capture and present quiz performance metrics in dashboards for both roles.

3. System Architecture

Quizzy follows a client-server architecture:

- **Frontend:** Vue.js 3 Single-Page Application styled with Bootstrap 5.
- **Backend:** Flask RESTful API, organized into modular blueprints for authentication, administration, and user operations.
- **Data Store:** SQLite database (quizzy.db) via SQLAlchemy ORM.
- **Asynchronous Processing & Caching:** Celery with Redis broker/backend, plus Flask-Caching for response caching.
- **Authentication & Security:** JSON Web Tokens (JWT) with Flask-JWT-Extended; rate limiting via Flask-Limiter.

4. Technology Stack

Layer	Technology	Layer	Technology
Backend	Python, Flask, SQLAlchemy	Frontend	JavaScript, Vue 3, Bootstrap 5
Database	SQLite	Authentication	JWT (Flask-JWT-Extended)
Asynchronous	Celery, Redis	Email Delivery	Flask-Mail (SMTP)

5. Backend Implementation

5.1 API Endpoints

- **Authentication (/api/auth):**
 - POST /register: User registration (rate-limited).
 - POST /login: Issue access token.
- **Administration (/api/admin):**
 - CRUD operations for subjects, chapters, quizzes, and questions.
 - Analytics: fetch score distributions and activity logs.
- **User (/api/user):**
 - GET /dashboard: Profile and quiz-performance summary.
 - GET /subjects, /quizzes, /quiz/<id>: Content retrieval.
 - POST /quiz/<id>/attempt: Submit answers and receive results.
 - GET /export: Trigger CSV export via background task.

All endpoints enforce JWT verification and apply rate limits (e.g., 20 req/min).

5.2 Database Schema

Key models include:

- **User:** id, full_name, email, qualification, dob, is_admin.
- **Subject → Chapter → Quiz → Question:** Hierarchical foreign-key relationships.
- **Score & UserAnswer:** Store attempt data and individual responses.
- **ActivityLog:** Record administrative actions and user events.

5.3 Background Tasks

- **Reminder Task:** send_daily_quiz_reminders emails users when quiz deadlines approach.
- **Export Task:** export_quiz_history_task compiles a CSV of a user's quiz history and emails it.
- Tasks are scheduled via Celery's beat scheduler and configured in celery_app.py.

6. Frontend Implementation

- **Routing:** Vue Router defines public (Landing, Login, Register) and protected (AdminLayout, UserLayout) routes.
- **State Management:** Pinia store (counter.js) for shared state (extendable to user/session data).
- **Components:** Separate Vue components for dashboards, list/detail views, quiz-taking workflow, and reporting.
- **Styling:** Bootstrap 5 classes ensure responsive layouts; form validation provides immediate feedback.

7. Setup & Installation

1. Backend

- Install Python dependencies: `pip install -r requirements.txt`.
- Ensure Redis is running on localhost:6379.
- Start Celery worker & scheduler:

```
celery -A celery_app worker --loglevel=info --pool=solo
```

```
celery -A celery_app beat --loglevel=info
```
- Activate the virtual environment : `.\venv\Scripts\Activate`
- Run Flask server: `python app.py` (default port 5000).

2. Frontend

- Install Node modules: `npm install` in frontend directory.
- Run dev server: `npm run dev` (default port 5173).

8. Challenges & Future Improvements

- **Scalability:** Migrate from SQLite to a production RDBMS (e.g., PostgreSQL).
- **Security Enhancements:** Introduce token refresh, stronger password policies, and HTTPS enforcement.
- **Real-Time Features:** Integrate WebSockets (Socket.IO) for live quiz timers and notifications.
- **Analytics Dashboard:** Add graphical visualizations of performance trends using charting libraries.

9. Conclusion

Quizzzy demonstrates a cohesive full-stack solution combining Python and JavaScript ecosystems to deliver a robust, extensible exam-prep platform. Its modular design and asynchronous capabilities lay the foundation for future scaling and feature expansion.

9. Video

Video drive link: https://drive.google.com/file/d/1L8agTB165TgvvHQ8jBfOD1Mx_4D4jsFr/view?usp=sharing